

AI 编译器学习计划 · 总览

面向 4 名硕士新生（基本零基础）的 AI 编译器学习计划，从现在开始、按阶段推进（参考节奏约 12 周，不绑定学期）。采用 **方案 B**：两条线并行，各 2 人，用我们自己的两块硬件实战。**主线**：两块卡都跑通 **resnet18 + llama2-7b**，读懂编译器，做小优化并用真卡数据验证效果。

两条学习线

学习线	计划文件	仓库	硬件	编译器类型
A 组 (2 人)	tpu-mlir-学习计划.md	sophgo/tpu-mlir	算能 BM1684X	图/模型级编译器 (ONNX→bmodel, 量化、LayerGroup)
B 组 (2 人)	tilelang-学习计划.md	tile-ai/tilelang	NVIDIA A100	kernel 级 DSL/编译器 (GEMM/Attention, Tensor Core)

两条线学的是 AI 编译器的两端：A 组看"整个模型如何被编译调度"，B 组看"单个 kernel 如何榨干硬件"。学完互讲，4 人对 AI 编译器有完整认识。

4 人分组

- A 组 (tpu-mlir)：组内再按方向分 —— 一人偏量化、一人偏 **LayerGroup/切分**。
- B 组 (tilelang)：组内分 —— 一人偏 kernel 实现、一人偏 **编译器 pass**。
- 每条线 2 人结对 debug，降低零基础卡死劝退的风险。

共用与协作（别漏）

- 第 1 周基础共用**：Linux / Git / Python / C++ 入门 4 人一起上，不讲两遍。
- 每周组会两条线一起开**：轮流给对方讲本周搞懂的东西。每人只深入一种范式，但不至于完全不懂另一半。
- 硬性 gate**：每份计划的阶段 0+1 / 2 / 4 是卡点，过不了不往下走，宁慢勿崩。

统一的 4 阶段结构（两份计划相同，约 12 周）

阶段	周次	内容
0+1	1-2	基础速成 + 编译器全局心智模型 + 跑通工具链（压缩到 2 周，边用边补）
2	3-6	在真实硬件上跑通 resnet18 + llama2-7b baseline，建立 benchmark
3	7-8	读编译器源码（带真实代码路径）， 定位优化点
4	9-12	做 小优化 ，真卡测 before/after，验证效果

每个阶段都按"学什么 / 练 / 资源"逐项展开，并附硬性 gate。

两个模型的角色：**resnet18** = 热身、吃透流程；**llama2-7b** = 真正的难点（TPU 侧走权重量化逐 block 编译，GPU 侧用 TileLang kernel 替换 attention/GEMM 热点）。

现实预期

零基础阶段性目标 = 每人能在真实硬件上跑通 **resnet18 + llama2-7b**，并完成一处经真卡数据验证的小优化。可发表的研究级贡献打好这套基础后再推进。

一条贯穿的研究线（对接 AIACC）

- A 组做 **LayerGroup/切分** 的学生 + B 组做 **编译器 pass** 的学生，本质都在解决同一类问题：片上内存受限下的切分 + 数据流/流水调度。
- 让这两人组成小组，做 GPU 侧（**cp.async pipeline**）与 TPU 侧（**LayerGroup**）的切分调度对照。
- 这是最可能接到 AIACC deterministic-streaming 方向、产出研究的路径。

怎么开始

1. 先按各自计划的 **阶段 0+1**（2 周）把环境搭好、建立全局心智模型。
2. A 组：跑通 tpu-mlir docker；B 组：在 A100 上跑通朴素 + tiled CUDA GEMM 和 TileLang 示例。
3. 过了阶段 0+1 的 gate，进阶段 2 开始跑 **resnet18 + llama2-7b**。

AI 编译器学习计划（tpu-mlir + 算能 BM1684X）

对象：本组 2 名硕士新生（方案 B 的 A 组），基本零基础 主线：在 BM1684X 上跑通 **resnet18 + llama2-7b**，读懂编译器，做小优化并用真卡数据验证效果 节奏：从现在开始、按阶段推进；下面的"周"是相对起点的参考节奏，不绑定学期

现实预期（先读这段）

零基础起步，目标定为**应用 + 入门级优化**，不强求改出可发表的成果：

- **目标**：每人能在 BM1684X 上独立跑通 resnet18 和 llama2-7b，并完成**至少一处小优化**，用真卡 before/after 数据说清效果。
- **研究级贡献**：打好上述基础后再推进，重点方向 = LayerGroup / 切分调度（见下）。
- **原则**：宁慢勿崩，阶段 0+1 / 2 / 4 是硬卡点，过不了不往下走。
- **关于压缩**：阶段 0+1 只给 2 周，对零基础偏紧。策略是"够用就走、边用边补"——靠工具链先跑起来，基础在后续阶段回填。

BM1684X 关键事实（决定学习重点）

- 算力约 INT8 32 TOPS / FP16 16 TFLOPS → **量化是部署主线**（resnet18 用 INT8，llama2-7b 用权重量化）。
- 架构：**NPU lane（并行计算单元）+ Local Memory + GDMA**。大 tensor 放不进片上 local memory，必须**切分**后搬进搬出。
- tpu-mlir 里负责切分的是 **LayerGroup pass**：决定算子怎么切块、在 local memory 上流水。
- 这是 BM1684X 编译器性能的命门，也是**最值得做研究的地方**。
- 与组里 AIACC deterministic-streaming 方向高度相关（都是"数据如何在片上 streaming + 切分调度"）。
- **llama2-7b 注意**：7B 权重 FP16 约 14GB，必须做**权重量化（W8A16 / W4A16）**才能高效部署；LLM 走的是逐 block 编译 + 权重量化的流程，和 resnet18 的单模型流程不同。

2 人分工

阶段 0+1、阶段 2（跑模型）一起做。从阶段 3 起按方向分头钻，组会互讲。

学生	方向
A1	量化 / 模型方向：calibration、INT8/W4/W8、精度恢复、llama2 量化

学生	方向
A2	LayerGroup / 编译器方向：切分调度、lowering、codegen ← 最贴研究方向

时间线（约 12 周）

周	阶段	内容	硬性产出（gate）
1–2	0+1	基础速成 + 编译器全局心智模型 + 跑通 docker	编译跑通 tpu-mlir docker；画出 ONNX→bmodel 流程图
3–6	2	跑通 resnet18 + llama2-7b 端到端（含量化），建立 benchmark	两个模型在真卡上的 baseline 精度/性能报告
7–8	3	读源码，定位“优化点在哪”	一篇源码走读笔记 + 指出可优化的瓶颈
9–12	4	做小优化，真卡测 before/after	一处优化 + 验证效果的报告/PR

阶段 0+1：基础速成 + 心智模型（第 1–2 周）

零基础压缩到 2 周，目标只有两个：环境能跑、脑子里有全局图。逐项给 学什么 / 练 / 资源。

第 1 周：工具链 + 基础 + 跑通 docker

☐ Linux + Git + Python

学：命令行、ssh、看进程/显存、Git clone/branch/commit、numpy 基本操作

练：纯命令行下载/解压/跑脚本；fork tpu-mlir 建分支

资源：The Missing Semester (missing.csail.mit.edu)；Pro Git 中文版

☐ 现代 C++ 够用（后面读源码要用，先建立印象，不求精通）

学：指针/引用、class、模板、unique_ptr、RAII、std::vector

资源：learncpp.com（按需查）

☐ 跑通官方 docker

练：拉起 tpu-mlir 官方 docker，编译成功，把一个示例模型跑通

这是本周硬指标，跑不通先解决环境

第 2 周：编译原理最小集 + 编译器全局心智模型

☐ 编译原理最小集（够用即可）

学：IR、SSA、basic block、pass / pass pipeline；核心心智：编译器 = 一串对 IR 做变换的 pass

资源：MLIR Toy Tutorial Ch.1–2 (mlir.llvm.org/docs/Tutorials/Toy)

☐ 深度学习 + 模型常识

学：conv / matmul / attention 在算什么；模型 = graph of ops；**transformer / decoder (llama2 是 decoder-only)** 的基本结构、什么是 prefill / decode

练：用 netron 打开 resnet18 的 ONNX，数清算子；再看一张 transformer block 结构图

资源：netron.app；任意一份 transformer 入门讲义

☐ tpu-mlir 编译流程心智模型

学：分层 框架模型 → Top 方言(硬件无关) → Top→Tpu lowering → Tpu 层优化/LayerGroup → Codegen → bmodel；渐进式 lowering 为什么分多层

Gate (阶段 0+1 硬卡点)

- ☐ tpu-mlir docker 编译跑通 + 一个示例模型出结果。
- ☐ 每人画一张 ONNX→bmodel 的流程图，能讲清主要阶段。

阶段 2：跑通 resnet18 + llama2-7b (第 3—6 周，上 BM1684X)

主线：两个模型都在真卡上跑出 baseline，并建立可复用的 benchmark 方法。

第 3 周：resnet18 端到端 (热身，吃透量化流程)

参考流程 (参数以官方文档/版本为准)：

```
# 转换: ONNX -> Top mlir
model_transform --model_name resnet18 --model_def resnet18.onnx \
  --input_shapes [[1,3,224,224]] --mlir resnet18.mlir
# F16 部署
model_deploy --mlir resnet18.mlir --quantize F16 --chip bm1684x --model resnet18_f16.bmodel
# INT8: 校准 + 部署
run_calibration resnet18.mlir --dataset ./calib_images --input_num 100 -o resnet18_calib_table
model_deploy --mlir resnet18.mlir --quantize INT8 --calibration_table resnet18_calib_table \
  --chip bm1684x --model resnet18_int8.bmodel
```

- ☐ 动手：F16 与 INT8 都部署到 BM1684X，和 CPU(ONNXRuntime)结果对拍
- ☐ 学：校准为什么要统计激活范围；预处理(mean/scale)对不上会全错
- ☐ 产出：resnet18 在 F32/F16/INT8 下的 Top-1 精度 + 单张延迟 + 吞吐

第 4—5 周：llama2-7b 编译部署 (重点、难点)

LLM 不走 resnet18 的单模型流程，而是 逐 block 编译 + 权重量化。请参考算能官方 LLM 部署示例 / 文档 (LLM-TPU / sophon) 作为脚手架，别从零造。

- ☐ 学：为什么 7B 必须权重量化(W8A16 / W4A16)才能放下、跑得动；LLM 的 prefill 与 decode 两个阶段
- ☐ 动手：把 llama2-7b 导出 → 用 tpu-mlir 编译成 bmodel (W8 或 W4 权重量化) → 在 BM1684X 上跑通文本生成
- ☐ 产出：llama2-7b 能在板上生成连贯文本（先求"跑通且正确"，性能下周测）
- ☐ 常见坑：显存/内存放不下、tokenizer/对齐、量化后乱码——逐项排查

第 6 周：建立 benchmark + 整理 baseline 报告

- ☐ resnet18 指标：精度(Top-1)、单张延迟、吞吐(img/s)
- ☐ llama2-7b 指标：首 token 延迟(prefill)、每 token 延迟 / tokens/s(decode)、生成质量(可用一小段 prompt 或 ppl 粗看)
- ☐ 动手：把测量脚本化，保证 before/after 可复现（固定输入、warmup、多次取中位数）
- ☐ 产出：两个模型在 BM1684X 上的 baseline 报告（resnet18 三档量化对比；llama2-7b W8 vs W4 的精度/速度），为阶段 4 的优化提供对照基线

Gate（本阶段硬卡点）

- ☐ resnet18 与 llama2-7b 都在真实 BM1684X 上跑通并有完整 baseline 数据。
- ☐ 有一套可复现的 benchmark 脚本。

阶段 3：读源码，定位优化点（第 7—8 周）

从"用户"转为"开发者"。目标不是读全部，而是找到 baseline 里最值得优化的地方。路径基于 sophgo/tpu-mlir 主分支（以实际为准）。

前端在 python/，编译器核心在 include/ (声明) + lib/ (实现)。一条算子生命周期：前端转换 → Top 方言 → Top→Tpu lowering → Tpu 层优化/LayerGroup → Codegen。

第 7 周：MLIR 基础设施 + 前端 + 一个算子的生命周期

- ☐ MLIR 基础设施：Operation/Region/Block、TableGen(.td)、Pass/PassManager、Pattern Rewrite。读 Toy Tutorial Ch.3—6
- ☐ 前端入口：python/tools/model_transform.py → python/transform/OnnxConverter.py、BaseConverter.py、MLIRImporter.py
- ☐ 挑一个算子(Conv 或 MatMul/Attention)逐文件追：
Top 定义：include/tpu_mlir/Dialect/Top/IR/TopOps.td；接口实现
lib/Dialect/Top/Interfaces/Conv.cpp
规范化/融合：lib/Dialect/Top/Canonicalize/Conv.cpp；芯片图优化
lib/Dialect/Top/Transforms/ProcessorOptimize/OptimizeBM1684X.cpp
lowering：lib/Conversion/TopToTpu/BM1684X/Conv.cpp（对比 F16 vs INT8 路径）

第 8 周：量化路径 (A1) + LayerGroup/Codegen (A2)，分头读

- ☐ A1 — 量化路径：python/calibration/ —— kld_calibrator.py、mix_precision.py、search_qtable.py

动手：找出 llama2-7b / resnet18 里对量化最敏感的层（阶段 4 优化候选）

- ☐ A2 — LayerGroup + Codegen：lib/Dialect/Tpu/Transforms/LayerGroup/ —— GroupMethod.cpp (分组策略)、LmemAllocator.cpp (local memory 分配)、TimeStepMethod.cpp / BasicTimeStep.cpp (时间步流水)、CycleCalculator.cpp (cost model); Codegen/地址 Codegen/BM168xCodegen.cpp、AddressAssign.cpp

动手：打开 LayerGroup debug 输出，看 resnet18 / llama2 的某段被怎么分组、哪里是瓶颈

- ☐ 组会互讲：A1 和 A2 各把自己读的那条线讲给对方

Gate（本阶段硬卡点）

- ☐ 每人一篇源码走读笔记（一条算子从前端到 codegen）。
- ☐ 明确指出阶段 4 要优化什么：一个具体的瓶颈或可改进点（敏感层量化 / 某融合 / LayerGroup 切分）。

阶段 4：小优化 + 验证效果（第 9—12 周）

核心 = 选一个小优化，做出来，用真卡 before/after 数据证明有效。从易到难，2 人分头做。

第 9—10 周：入门优化（人人都做）

- ☐ 选一个低风险 knob 做优化并测效果，例如：

resnet18：试 per-channel 量化 / 混合精度，测精度+延迟

llama2-7b：W4 vs W8 权重量化对比，测 tokens/s + 生成质量

加一个图融合 pattern (Canonicalize/Op.cpp 或 ProcessorOptimize/OptimizeBM1684X.cpp)

- ☐ 产出：优化前后的指标对比表 + 一句话归因

第 11—12 周：分方向进阶

- ☐ A1 (量化/模型)：把 llama2-7b 的敏感层用更高精度保护、其余更激进量化，目标“几乎不掉质量但更快/更小”

入手：python/calibration/ (mix_precision.py、search_qtable.py)

- ☐ A2 (LayerGroup/编译器) ← 最贴研究：改 LayerGroup 分组或 local memory 分配策略，测延迟/吞吐 before/after

入手：LayerGroup/GroupMethod.cpp、LmemAllocator.cpp、CycleCalculator.cpp

Gate（本阶段硬卡点 = 收尾 capstone）

- ☐ 每人一处优化，真卡 before/after 数据证明效果（精度不退、或速度/显存改善），形成报告/内部 PR。

- ☐ 报告要有归因：为什么这个优化有效、瓶颈是否真的被缓解。
-

贯穿全程的资源

- MLIR 官方文档 + Toy Tutorial (dialect 与 lowering 入门)
 - tpu-mlir 官方文档与技术报告；算能 LLM-TPU / sophon 的 LLM 部署示例 (llama2 部署脚手架)
 - TVM / XLA 设计文档作对照
-

运行机制（零基础尤其重要）

- 每周组会轮讲：逼输出，讲不清就是没懂；两条线（tpu-mlir / TileLang）合并开，互相讲
- 结对：2 人一起 debug
- 硬卡点：阶段 0+1 / 2 / 4 的产出为硬性 gate，过不了不往下
- 收尾 capstone：每人在 BM1684X 上跑通 resnet18 + llama2-7b，并完成一处经真卡数据验证的小优化

AI 编译器学习计划（TileLang + NVIDIA A100）

配套：见同目录 tpu-mlir-学习计划.md（图/模型级编译器） 本计划：kernel 级 DSL + 编译器，目标 NVIDIA A100（Ampere）Tensor Core 方案：**方案 B 并行轨**——本组 2 名学生，与 tpu-mlir 组并行推进 主线：在 A100 上跑通 **resnet18 + llama2-7b**，用 TileLang kernel 优化热点，验证优化效果 节奏：从现在开始、按阶段推进；下面的"周"是相对起点的参考节奏，不绑定学期。学生零基础起步

TileLang 是什么 / 和 tpu-mlir 的区别

维度	tpu-mlir	TileLang
层次	图/模型级 AOT 编译器	kernel 级 DSL + 编译器
输入	整个模型（ONNX→bmodel）	手写的单个高性能 kernel（GEMM、FlashAttention...）
在模型里的角色	编译整个模型	替换推理管线里的热点 kernel（attention / GEMM）
目标硬件	算能 TPU（BM1684X）	NVIDIA A100 / Tensor Core
"改编译器"改什么	lowering / 量化 / LayerGroup	tile 原语、layout 推断、pipeline、Tensor Core lowering

关键差异：TileLang 不像 tpu-mlir 那样"编译整个模型"。本计划里跑 resnet18 / llama2-7b 的方式是：用 PyTorch/HF 跑模型作 baseline → profile 找热点 → 用 TileLang 写的 kernel 替换热点 → 测 before/after。

运行方式：方案 B（已确定）

- 本组 2 名学生跟 TileLang / A100，另 2 名跟 tpu-mlir，**并行推进**。
- 对策：组内 2 人结对 debug；每周组会两条线一起开，互相讲。

目标硬件：NVIDIA A100（Ampere, sm_80）

- 算力：FP16/BF16 Tensor Core ~312 TFLOPS、FP32 ~19.5 TFLOPS、支持 **TF32**。
- 片上资源：每 SM 最多 ~164KB 可用 shared memory（远大于消费卡）→ tile 可开更大，是调优关键。
- Ampere 专属机制（TileLang 都会用到）：

- `cp.async` : global→shared 异步拷贝, 是 pipeline / double buffering 的基础。
- `mma.sync` (如 m16n8k16): Ampere 的 Tensor Core 指令 (不是 Hopper 的 wgmma/TMA)。
- `shared memory swizzle`: 避免 bank conflict 的 layout 技巧。
- 调优主线 = 大 tile + `cp.async` 多级 pipeline + `mma.sync` + `swizzle`。

现实预期 (先读这段)

- 目标: 每人能在 A100 上跑通 resnet18 + llama2-7b, 用 TileLang kernel 替换至少一个热点, 并用 before/after 数据证明效果。
- 关于压缩: 阶段 0+1 只给 2 周, 对零基础 CUDA 偏紧。策略是"先跑起来、边用边补", CUDA 细节在后续阶段回填。

时间线 (约 12 周)

周	阶段	内容	硬性产出 (gate)
1—2	0+1	CUDA 基础速成 + TileLang 心智模型 + 跑通官方示例	跑通朴素+tiled CUDA GEMM; 跑通 TileLang GEMM 示例
3—6	2	A100 上跑 resnet18 + llama2-7b baseline, 用 TileLang kernel 替换热点	两模型 baseline + 至少一个热点被 TileLang kernel 替换并验证
7—8	3	读 TileLang 编译器内部, 定位 kernel 性能来源	一篇 tile 原语走读笔记 + 指出可优化点
9—12	4	做小优化 (调 kernel / 改 pass), A100 测 before/after	一处优化 + 验证效果的报告/PR

阶段 0+1: CUDA 速成 + TileLang 心智模型 (第 1—2 周)

零基础压缩到 2 周, 目标: 会写最基本的 GPU kernel + 跑通 TileLang 示例。逐项给 学什么 / 练 / 资源。

第 1 周: CUDA 编程模型 + 内存层次 (写出 GEMM)

☐ 工具 + 执行模型

学: `ssh / nvidia-smi / Git`; grid/block/thread/warp、kernel 启动 `<<<<>>>`

练: 写 vector add, 再写一个朴素 GEMM (每 thread 算一个输出元素), 在 A100 测 GFLOPS

☐ 内存层次 + 性能

学: global / **shared memory** / register、**coalescing**、bank conflict、occupancy

练: 把朴素 GEMM 改成 **shared memory tiled** 版, 重测 GFLOPS, 对比提升

资源: CUDA C++ Programming Guide (docs.nvidia.com/cuda/cuda-c-programming-guide)

第 2 周: Tensor Core / Ampere 概念 + TileLang 心智模型

☐ Tensor Core / Ampere (建立印象即可)

学: 为什么矩阵乘要用 Tensor Core; FP16/TF32 输入+FP32 累加; `mma.sync`、`cp.async`、`swizzle` 各是干什么的

☐ TileLang 心智模型 + 跑示例

学: 核心抽象 `T.Kernel`、`T.alloc_shared` / `T.alloc_fragment`、`T.copy`、`T.gemm`、

`T.Pipelined`; 分工——你写 tile 计算, 编译器负责 layout/内存/pipeline/Tensor Core lowering

练: clone 仓库, 跑通官方 `examples/gemm` 和 `examples/flash_attention`, 在 A100 上测性能、和 cuBLAS 比

资源: TileLang README (github.com/tile-ai/tilelang); TVM TIR 文档 (tvm.apache.org/docs)

Gate (阶段 0+1 硬卡点)

☐ 朴素 + tiled CUDA GEMM 都在 A100 跑通并测出 GFLOPS, 能说清快慢原因。

☐ 跑通 TileLang 官方 GEMM / FlashAttention 示例并记录性能。

阶段 2: 跑通 resnet18 + llama2-7b (第 3-6 周, 上 A100)

主线: 两个模型在 A100 上跑 baseline, profile 找热点, 用 TileLang kernel 替换并验证。

第 3 周: resnet18 (热身, 掌握"替换 kernel"方法论)

☐ 动手: 用 PyTorch 在 A100 跑 resnet18 推理作 baseline, profile 出热点算子 (conv / 末端 GEMM)

☐ 动手: 用 TileLang 写一个 conv 或 GEMM kernel (参考 `examples/convolution`、`examples/gemm`), 替换该热点

☐ 产出: 替换后数值正确性对拍通过 + 该算子 before/after 耗时; 建立"profile→替换→验证"的标准流程

第 4 周: llama2-7b baseline + 找热点

☐ 学: llama2 是 decoder-only transformer; prefill 与 decode 两阶段; 热点 = attention + 各 linear 层 GEMM

☐ 动手: 用 HF/PyTorch 在 A100 跑 llama2-7b (FP16) 推理 baseline, 能生成连贯文本

☐ 动手: profile 出 attention 和 GEMM 占了多少时间 (decode 阶段尤其受 GEMM/访存影响)

☐ 产出: llama2-7b baseline 的 tokens/s、prefill/decode 延迟 + 热点占比

第 5 周: 用 TileLang kernel 替换 llama2 热点

☐ 动手: 把 attention 换成 TileLang FlashAttention (参考 `examples/flash_attention`), 或把某个 linear 换成 TileLang GEMM, 接进推理管线

- ☐ 产出：替换后生成质量不变 + 该热点的 before/after 耗时

第 6 周：建立 benchmark + 整理 baseline

- ☐ 指标：resnet18(延迟/吞吐/精度)；llama2(tokens/s、prefill/decode 延迟、生成质量)
- ☐ 动手：测量脚本化，保证可复现（固定输入、warmup、多次取中位数）
- ☐ 产出：两模型 baseline 报告 + 已替换热点的初步收益

Gate（本阶段硬卡点）

- ☐ resnet18 与 llama2-7b 都在 A100 上跑通并有 baseline 数据。
- ☐ 至少一个热点 kernel 已用 TileLang 替换，正确性通过、有 before/after 数据。

阶段 3：读 TileLang 编译器内部，定位优化点（第 7–8 周）

目标不是读全部，而是搞清你替换的 kernel 性能来自哪里、还能往哪优化。路径基于 tile-ai/tilelang 主分支（以实际为准）。

结构：Python 前端在 `tilelang/`，编译器核心(C++)在 `src/`。流程：Python DSL(`tilelang/language`) → tile IR(`src/op`) → transform pass(`src/transform`) → codegen(`src/target / src/tl_templates`) → CUDA。A100=sm_80，跳过带 `hopper / sm90 / tcgen05 / blackwell / sm100` 的文件。

第 7 周：前端 DSL + tile 原语 + 核心 transform pass

- ☐ Python 前端： `tilelang/language/` (`kernel.py`、`gemm_op.py`、`copy_op.py`、`loop.py`、`reduce_op.py`)
- ☐ C++ 原语： `src/op/` (`gemm.cc`、`copy.cc`、`reduce.cc`) ——读 `gemm.cc` 看 `T.gemm` 在 IR 层是什么
- ☐ 核心 transform pass (目录 `src/transform/`，都对应你 kernel 的性能)：
`layout / swizzle`: `layout_inference.cc` + `src/layout/gemm_layouts.cc`、`layout.cc`
`pipeline`: `pipeline_planning.cc`、`inject_pipeline.cc`、`multi_version_buffer_rewriter.cc`
`cp.async`: `lower_ptx_async_copy.cc`、`ptx_async_copy_injector.h`
`shared memory 规划`: `merge_shared_memory_allocations.cc`、`storage_rewrite.cc`
- ☐ 动手：dump 你的 kernel 各 pass 前后 IR，看 layout/pipeline 怎么被加进去

第 8 周：codegen → CUDA + 定位优化点

- ☐ codegen: `src/target/` (`codegen_c.cc`)；CUDA 模板: `src/tl_templates/cuda/` (`gemm.h`、`copy.h`、`common.h`)
- ☐ 动手：dump 你 kernel 生成的 CUDA，对照看 `mma`、`cp.async` 落在哪；用 `ncu` 定位瓶颈（访存？bank conflict？占用？）

Gate（本阶段硬卡点）

- ☐ 每人一篇 tile 原语走读笔记（DSL → op → transform → CUDA）。
- ☐ 明确指出阶段 4 要优化什么：一个具体瓶颈（tile 尺寸 / pipeline 级数 / swizzle / 某 pass 行为）。

阶段 4：小优化 + 验证效果（第 9–12 周）

核心 = 选一个小优化，做出来，用 A100 before/after 数据证明有效。从易到难，2 人分头做。

第 9–10 周：入门优化（人人都做）

- ☐ 调你替换进 llama2/resnet18 的那个 kernel 的配置并测效果：
tile 尺寸扫描、pipeline 级数(2/3/4)、加 swizzle、用满 164KB shared memory、autotune（`tilelang/autotuner/`）
- ☐ 产出：优化前后 kernel 耗时 + 端到端(tokens/s 或延迟)对比 + 一句话归因

第 11–12 周：分方向进阶

- ☐ 学生 1 (kernel 方向)：实现/优化一个更难的 kernel
目标：FlashAttention 变体 或 dequant GEMM（对接 llama2 量化权重），对标 baseline
入手：`examples/flash_attention/`、`examples/dequantize_gemm/` + `tilelang/language/`
- ☐ 学生 2 (编译器方向) ← 最贴研究：改一个 pass，A100 测 before/after
pipeline: `src/transform/pipeline_planning.cc` / `inject_pipeline.cc`
layout/swizzle: `src/layout/gemm_layouts.cc` / `src/transform/layout_inference.cc`（用 `ncu` 看 bank conflict 降没降）
cp.async: `src/transform/lower_ptx_async_copy.cc`
注意：只动 Ampere/sm_80 路径，别碰 hopper/tcgen05/blackwell

Gate（本阶段硬卡点 = 收尾 capstone）

- ☐ 每人一处优化，A100 before/after 数据证明效果（kernel 提速 + 端到端可见收益），形成报告/内部 PR。
- ☐ 报告要有归因：提升来自哪（更好的 pipeline / 更少 bank conflict / 更高占用）。

资源

- TileLang README + examples（gemm / flash_attention / dequantize_gemm / convolution）
 - CUDA C++ Programming Guide + Best Practices Guide + Nsight Compute 文档
 - TVM TIR 文档；FlashAttention 论文（理解融合 kernel 的算法动机）
-

与 tpu-mlir 计划的衔接

- 相同运行机制：每周组会**两条线合并开**、结对、阶段 0+1/2/4 硬卡点。
- 研究衔接：TileLang 的 **tile 切分 + pipeline 调度** 与 tpu-mlir 的 **LayerGroup**，本质都是"片上内存受限下的切分与数据流编排" → 对接 AIACC deterministic-streaming 方向。
- 建议：TileLang 组做编译器方向的学生 + tpu-mlir 组做 LayerGroup 的学生，组成"切分/数据流"研究小组，做 GPU 侧 (cp.async pipeline) 与 TPU 侧 (LayerGroup) 的对照——最可能接到 AIACC 论文的路径。

Capstone 验收标准 + 评分表

适用于两条线（tpu-mlir / TileLang）整套做完后的验收考核。两条线对齐到同一套维度，便于把 4 个学生放在同一尺度比较。

一、Capstone 任务（两条线对应）

每人需完成 三件事：① 跑通两个模型 ② 一处小优化 ③ 一份报告。

	A 组（tpu-mlir / BM1684X）	B 组（TileLang / A100）
① 跑通模型	resnet18 + llama2-7b 都部署到 BM1684X 跑通（含量化），有 baseline	resnet18 + llama2-7b 在 A100 跑通 baseline，至少一个热点用 TileLang kernel 替换
② 小优化	选一处优化（量化策略 / 融合 / LayerGroup），真卡测 before/after	调 kernel 配置或改一个 pass，A100 测 before/after
③ 报告	含真卡数据 + 优化前后对比 + 归因	含 A100 数据 + kernel/端到端 before/after + 归因

及格线 vs 优秀线

	及格（"会跑 + 看懂"）	优秀（"能优化 + 说清为什么"）
跑通模型	两个模型都跑通、结果正确，有 baseline	baseline 完整规范，热点定位准确
小优化	做出一处优化、不破坏正确性	优化有实测收益，且能解释收益来源
报告	有数据、流程清晰	有归因分析、指出瓶颈与下一步，能对接研究方向

二、评分表（满分 100）

每个维度按 4 档给分，乘以权重。

维度	权重	不及格	及格	良好	优秀
模型跑通完成度（resnet18 + llama2-7b baseline）	25%	跑不通	两个都跑通、结果正确	baseline 完整规范	热点定位准确、benchmark 可复现
源码理解深度（阶段 3 走读笔记）	20%	说不清流程	能讲清一条算子/原语的主干路径	能讲清关键 pass 做了什么	能批判性指出设计取舍

维度	权重	不及格	及格	良好	优秀
优化质量（阶段 4）	25%	无有效优化	做出优化、不破坏正确性	优化正确且有收益	收益明显且归因清晰
真卡数据与归因	20%	无数据	有 before/after 数据	数据完整、对比清晰	有瓶颈分析与下一步
组会表达与协作	10%	很少参与	能讲清自己的工作	能给另一条线讲懂	主动帮助同伴、推动协作

档位对应分数（乘权重前）：不及格 = 0—59，及格 = 60—74，良好 = 75—89，优秀 = 90—100。

计分示例

某学生：端到端 良好(82) ×25% + 源码 及格(70) ×20% + 改动 良好(80) ×25% + 数据 优秀(92) ×20% + 表达 良好(80) ×10% = 20.5 + 14.0 + 20.0 + 18.4 + 8.0 = **80.9（良好）**

三、过程性硬卡点（不计分，但不过不能进下一阶段）

来自各自计划的阶段 gate，作为资格门槛而非打分项：

- 阶段 0+1 gate：环境搭好 + 全局心智模型（A：跑通 docker、画出 ONNX→bmodel 流程图；B：跑通朴素+tiled CUDA GEMM 和 TileLang 示例）
- 阶段 2 gate：resnet18 + llama2-7b 都在真卡跑通、有 baseline（B 组另需至少一个热点已用 TileLang kernel 替换）
- 阶段 4 gate：一处优化 + 真卡 before/after 数据

任何一个 gate 未过，该生 capstone 不予验收，先补齐再考核。

四、研究加分项（可选，对接 AIACC）

做 LayerGroup（A 组）或编译器 pass（B 组）方向、且在报告里把"片上内存受限下的切分/流水调度"讲出门道的学生，额外认定为**研究苗子**，优先进入后续研究小组。